Python Lists — Cheat Sheet

Movie watchlist edition · 5 November 2026

List basics

A list is an ordered, changeable collection written in square brackets: it keeps duplicates and can hold items of different types.

Property	What it means	Example
Ordered	Items keep their position; the first item is index 0	`movies[0]`
Mutable	Items can be added, removed or changed in place	`movies[0] = "Memento"`
Allows duplicates	The same value can appear more than once	`["Jaws", "Jaws"]`
Mixed types	Strings, numbers, booleans and even lists can share one list	`[1, "two", 3.0, True]`

Five common ways to create a list:

```
movies = ["Inception", "Avatar", "Gladiator"]   # literal with square brackets
empty = []                                      # empty list
empty_too = list()                              # empty list via the constructor
letters = list("abc")                           # ['a', 'b', 'c']
words = "Matrix Alien Shrek".split()            # ['Matrix', 'Alien', 'Shrek']
```

Indexing and slicing

Square brackets pick items by position, negative numbers count from the end, and a slice returns a new list. The examples use `m = ["Titanic", "Rocky", "Jaws", "Inception", "Gladiator"]`.

Expression	Result	Meaning
`m[0]`	`'Titanic'`	First item
`m[-1]`	`'Gladiator'`	Last item
`m[1:3]`	`['Rocky', 'Jaws']`	From index 1 up to, not including, index 3
`m[:2]`	`['Titanic', 'Rocky']`	First two items
`m[2:]`	`['Jaws', 'Inception', 'Gladiator']`	From index 2 to the end
`m[::2]`	`['Titanic', 'Jaws', 'Gladiator']`	Every second item
`m[::-1]`	`['Gladiator', 'Inception', 'Jaws', 'Rocky', 'Titanic']`	Reversed copy; `m` itself is unchanged

Built-in lookups that work on any list:

Expression	Result	Meaning
`len(m)`	`5`	Number of items
`"Jaws" in m`	`True`	Membership test
`m.index("Jaws")`	`2`	Position of the first match; raises `ValueError` if missing
`m.count("Jaws")`	`1`	How many times the value appears
`min(m)`, `max(m)`	`'Gladiator'`, `'Titanic'`	Alphabetically first and last

Core list methods

Seven methods cover most day-to-day list work, and all of them change the list in place. Only `pop()` hands back a value; the other six return `None`.

Method	Syntax	What it does	Returns	Fails when
`append()`	`lst.append(x)`	Adds one item to the end	`None`	Never
`insert()`	`lst.insert(i, x)`	Puts `x` before index `i`; `0` means the front	`None`	Never; an index past the end adds at the end
`remove()`	`lst.remove(x)`	Deletes the first item equal to `x`	`None`	`x` is not in the list (`ValueError`)
`pop()`	`lst.pop()` or `lst.pop(i)`	Removes the last item, or the item at `i`	The removed item	The list is empty or `i` is out of range (`IndexError`)
`extend()`	`lst.extend(iterable)`	Adds every item of another list (or any iterable) to the end	`None`	The argument is not iterable (`TypeError`)
`sort()`	`lst.sort()`	Sorts ascending, in place	`None`	Items cannot be compared, such as numbers mixed with strings (`TypeError`)
`reverse()`	`lst.reverse()`	Flips the order, in place	`None`	Never

Each method in a row, building on the previous line:

```
lst = ["B", "C"]
lst.append("D")        # ['B', 'C', 'D']              add one item to the end
lst.insert(0, "A")     # ['A', 'B', 'C', 'D']         add at index 0
lst.remove("C")        # ['A', 'B', 'D']              delete the first "C"
last = lst.pop()       # last = 'D', lst = ['A', 'B']  remove and keep the last item
lst.extend(["Z", "Y"]) # ['A', 'B', 'Z', 'Y']         add many items at once
lst.sort()             # ['A', 'B', 'Y', 'Z']         alphabetical order
lst.reverse()          # ['Z', 'Y', 'B', 'A']         flipped order
```

Also handy: `lst.sort(reverse=True)` sorts descending, `sorted(lst)` returns a sorted copy and leaves `lst` alone, `lst.copy()` makes a shallow copy, and `lst.clear()` empties the list.

Operators and string conversion

`+` joins two lists, `*` repeats a list, and `split()` turns a string into a list. All three build a new object and leave the original untouched.

Operation	Syntax	Result
Concatenate	`a + b`	A new list holding the items of `a`, then the items of `b`
Repeat	`a * n`	A new list with the items of `a` repeated `n` times
String to list	`s.split()`	A list of words, splitting on any run of whitespace
String to list with a separator	`s.split(", ")`	A list split wherever the separator text appears
List to string (reverse of split)	`" ".join(lst)`	One string with the items joined by the separator

```
a = ["Titanic", "Rocky"]
b = ["Frozen", "Coco"]

a + b                                  # ['Titanic', 'Rocky', 'Frozen', 'Coco']
a * 2                                  # ['Titanic', 'Rocky', 'Titanic', 'Rocky']
"Matrix Alien Shrek".split()           # ['Matrix', 'Alien', 'Shrek']
"Matrix, Alien, Shrek".split(", ")     # ['Matrix', 'Alien', 'Shrek']
"  Matrix   Alien Shrek ".split()      # ['Matrix', 'Alien', 'Shrek']  extra spaces are ignored
" ".join(["Matrix", "Alien", "Shrek"])   # 'Matrix Alien Shrek'
```

Because `+` and `*` return new lists, store the result in a variable (`combined = a + b`) or it is lost.

Gotchas and tips

Most list bugs come from seven habits, and each has a one-line fix.

Mistake	What happens	Do this instead
`lst.append(["Jaws", "Rocky"])`	The whole list is added as one nested item: `[..., ['Jaws', 'Rocky']]`	`lst.extend(["Jaws", "Rocky"])`
`lst = lst.sort()`	`lst` becomes `None`, because `sort()` returns nothing	Call `lst.sort()` on its own line, or use `lst = sorted(lst)`
`lst.remove("Avatar")` when it is absent	`ValueError`	Check first: `if "Avatar" in lst:`
`lst.remove("Jaws")` with two copies of `"Jaws"`	Only the first copy is removed	`lst = [m for m in lst if m != "Jaws"]`
`b = a`	Both names point to the same list, so changing one changes the other	`b = a.copy()`
`lst.pop()` on `[]`	`IndexError`	Check first: `if lst:`
`["apple", "Banana"].sort()`	Uppercase sorts first: `['Banana', 'apple']`	`lst.sort(key=str.lower)`

Final program: movie watchlist

One script covers all 11 tasks and prints the result after each operation.

```
# Movie Watchlist - all 11 tasks in one program

# 1. Create the list
my_watchlist = ["Inception", "Avatar", "Gladiator"]
print("1. Initial watchlist:", my_watchlist)

# 2. append() - add to the end
my_watchlist.append("Interstellar")
print("2. After append:", my_watchlist)

# 3. insert() - add at the beginning (index 0)
my_watchlist.insert(0, "Titanic")
print("3. After insert:", my_watchlist)

# 4. remove() - delete the first matching value
my_watchlist.remove("Avatar")
print("4. After remove:", my_watchlist)

# 5. pop() - remove the last item and keep it
removed_movie = my_watchlist.pop()
print("5. Removed movie:", removed_movie)

# 6. extend() - add several items at once
my_watchlist.extend(["Jaws", "Rocky"])
print("6. After extend:", my_watchlist)

# 7. sort() - alphabetical order (in place)
my_watchlist.sort()
print("7. After sort:", my_watchlist)

# 8. reverse() - flip the order (in place)
my_watchlist.reverse()
print("8. After reverse:", my_watchlist)

# 9. + operator - concatenate into a NEW list
combined_watchlist = my_watchlist + ["Frozen", "Coco"]
print("9. After concatenation:", combined_watchlist)

# 10. * operator - repeat the list twice into a NEW list
duplicated_watchlist = my_watchlist * 2
print("10. After repetition:", duplicated_watchlist)

# 11. split() - convert a string into a list
movie_text = "Matrix Alien Shrek"
movie_list = movie_text.split()
print("11. Converted list:", movie_list)
```

Steps 9 and 10 use `+` and `*`, which return new lists, so `my_watchlist` still holds the five movies from step 8. Step 10 reads “duplicate twice” as `my_watchlist * 2`, which repeats the list two times in a row.

Program output

Running the program prints these 11 lines, copied from a real run on Python 3.

```
1. Initial watchlist: ['Inception', 'Avatar', 'Gladiator']
2. After append: ['Inception', 'Avatar', 'Gladiator', 'Interstellar']
3. After insert: ['Titanic', 'Inception', 'Avatar', 'Gladiator', 'Interstellar']
4. After remove: ['Titanic', 'Inception', 'Gladiator', 'Interstellar']
5. Removed movie: Interstellar
6. After extend: ['Titanic', 'Inception', 'Gladiator', 'Jaws', 'Rocky']
7. After sort: ['Gladiator', 'Inception', 'Jaws', 'Rocky', 'Titanic']
8. After reverse: ['Titanic', 'Rocky', 'Jaws', 'Inception', 'Gladiator']
9. After concatenation: ['Titanic', 'Rocky', 'Jaws', 'Inception', 'Gladiator', 'Frozen', 'Coco']
10. After repetition: ['Titanic', 'Rocky', 'Jaws', 'Inception', 'Gladiator', 'Titanic', 'Rocky',
    'Jaws', 'Inception', 'Gladiator']
11. Converted list: ['Matrix', 'Alien', 'Shrek']
```

Line 10 is wrapped here to fit the page; Python prints it on a single line.

Task-to-concept map

Each of the 11 tasks exercises one list concept, and together they cover every concept the exercise asks for.

Task	What the program does	Python concept
1	Create `my_watchlist` with three movies	List creation
2	Add `"Interstellar"` to the end	`append()`
3	Add `"Titanic"` at the front	`insert(0, x)`
4	Delete `"Avatar"`	`remove()`
5	Take off the last movie into `removed_movie`	`pop()`
6	Add `"Jaws"` and `"Rocky"` together	`extend()`
7	Alphabetical order	`sort()`
8	Flip the order	`reverse()`
9	Join with `["Frozen", "Coco"]`	Concatenation with `+`
10	Repeat the list twice	Repetition with `*`
11	Turn `"Matrix Alien Shrek"` into a list	`split()`